

Actividad - Proyecto práctico

<https://github.com/osoria80/O8MIAR-Proyecto-Programacion>

La actividad se desarrollará en grupos pre-definidos de 4 alumnos. Se debe indicar los nombres en orden alfabético (de apellidos). Recordad que esta actividad se corresponde con un 30% de la nota final de la asignatura. Se debe entregar el trabajo en la presente notebook.

- Alumno 1: German Ricardo Malnero
- Alumno 2: Guillermo de Lecea Ramos
- Alumno 3: Oscar Soria Corral
- Alumno 4: Rigel Chuliá Ortega

✓ PARTE 1 - Instalación y requisitos previos

Las prácticas han sido preparadas para poder realizarse en el entorno de trabajo de Google Colab. Sin embargo, esta plataforma presenta ciertas incompatibilidades a la hora de visualizar la renderización en gym. Por ello, para obtener estas visualizaciones, se deberá trasladar el entorno de trabajo a local. Por ello, el presente dossier presenta instrucciones para poder trabajar en ambos entornos. Siga los siguientes pasos para un correcto funcionamiento:

1. **LOCAL:** Preparar el environment, siguiendo las instrucciones detalladas en la sección *1.1.Preparar environment*.
2. **AMBOS:** Modificar las variables "mount" y "drive_mount" a la carpeta de trabajo en drive en el caso de estar en Colab, y ejecturar la celda *1.2.Localizar entorno de trabajo*.
3. **COLAB:** se deberá ejecutar las celdas correspondientes al montaje de la carpeta de trabajo en Drive. Esta corresponde a la sección *1.3.Montar carpeta de datos local*.
4. **AMBOS:** Instalar las librerías necesarias, siguiendo la sección *1.4.Instalar librerías necesarias*.

```
1 # Instalación de librerías necesarias
2
3 %pip install torch torchvision --index-url https://download.pytorch.org/
4 %pip install "gymnasium[atari,accept-rom-license]"
```

```

5 %pip install ale-py
6 %pip install numpy pillow matplotlib
7 %pip install tqdm
8

```

Looking in indexes: <https://download.pytorch.org/whl/cu121>

```

Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/py
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dis
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dis
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dis
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/py
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/li
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/pyt
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/p
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/l
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/lo
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/l
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/l
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/loca
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/loca
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93.* in /usr/loca
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/loca
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: pillow!>=8.3.*,>=5.3.0 in /usr/local/lib/python
Requirement already satisfied: cuda-pathfinder~1.1 in /usr/local/lib/python3
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.1
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: gymnasium[accept-rom-license,atari] in /usr/lo
WARNING: gymnasium 1.3.0 does not provide the extra 'accept-rom-license'
Requirement already satisfied: numpy>=1.21.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.1
Requirement already satisfied: typing-extensions>=4.3.0 in /usr/local/lib/pyt
Requirement already satisfied: farama-notifications>=0.0.1 in /usr/local/lib/
Requirement already satisfied: ale_py>=0.9 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: ale-py in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: numpy>1.20 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-package

```

```
1 import torch
2
3 print("PyTorch version:", torch.__version__)
4 print("CUDA disponible:", torch.cuda.is_available())
5 if torch.cuda.is_available():
6     print("GPU detectada:", torch.cuda.get_device_name(0))
7
8 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
9 print("Usando dispositivo:", device)
10
```

```
PyTorch version: 2.11.0+cu128
CUDA disponible: True
GPU detectada: NVIDIA L4
Usando dispositivo: cuda
```

PARTE 2. Enunciado

Consideraciones a tener en cuenta:

- El entorno sobre el que trabajaremos será el indicado en el listado correspondien de cada grupo y el algoritmo que usaremos será *DQN*.
- Para nuestro ejercicio, el requisito mínimo será alcanzado cuando el agente consiga una **media de recompensa por encima de los puntos indicados en el listado por grupos en modo test**. Por ello, esta media de la recompensa se calculará a partir del código de test en la última celda del notebook.

Este proyecto práctico consta de tres partes:

1. [Implementar la red neuronal que se usará en la solución](#)
2. Implementar las distintas piezas de la solución DQN y probar al menos 3 propuestas diferentes de mejora.
3. Justificar la respuesta en relación a los resultados obtenidos e incluir al menos 3 gráficas relevantes comparando las 3 propuestas.

Rúbrica: Se valorará la originalidad en la solución aportada, así como la capacidad de discutir los resultados de forma detallada. El requisito mínimo servirá para aprobar la actividad, bajo premisa de que la discusión del resultado sera apropiada.

IMPORTANTE:

- Si no se consigue una puntuación óptima, responder sobre la mejor puntuación obtenida.
- Para entrenamientos largos, recordad que podéis usar checkpoints de vuestros modelos para retomar los entrenamientos. En este caso, recordad cambiar los

parámetros adecuadamente (sobre todo los relacionados con el proceso de exploración).

- Se deberá entregar únicamente el notebook y los pesos del mejor modelo en un fichero .zip, de forma organizada.
- Cada alumno deberá de subir la solución de forma individual.

✓ PARTE 3. Desarrollo

> 3.1 - Preparación previa

↳ 10 celdas ocultas

> 3.2 - DQN

↳ 23 celdas ocultas

> 3.3 - Implementación de 3 mejoras propuestas

↳ 29 celdas ocultas

✓ 3.4 - Análisis, comparación de resultados y conclusiones

```
1 # =====
2 # COMPARATIVA COMPLETA: carga de modelos, test y gráficas de justificac
3 # =====
4
5 import os
6 import json
7 import numpy as np
8 import matplotlib.pyplot as plt
9
10 N_TEST_EPISODES = 20 # nº de episodios de test por modelo
11
12 MODELS_TO_COMPARE = [
13     {
14         "name": "DQN",
15         "agent_class": DQNAgent,
16         "model_class": DQNCNN,
17         "weights_file": f"dqn_{ENV_NAME.replace('/', '_')}_weights.pt",
18         "history_file": "history_dqn_SpaceInvaders.json",
19     },
20     {
21         "name": "DDQN",
```

```

22     "agent_class": DDQNAgent,
23     "model_class": DQNCNN,
24     "weights_file": f"ddqn_{ENV_NAME.replace('/', '_')}_weights.pt"
25     "history_file": "history_ddqn_SpaceInvaders.json",
26 },
27 {
28     "name": "Dueling DQN",
29     "agent_class": DuelingDQNAgent,
30     "model_class": DuelingDQNCNN,
31     "weights_file": f"dueling_{ENV_NAME.replace('/', '_')}_weights.pt"
32     "history_file": "history_dueling_SpaceInvaders.json",
33 },
34 {
35     "name": "DQN Tuned",
36     "agent_class": DQNAgent,
37     "model_class": DQNCNN,
38     "weights_file": f"tuned_{ENV_NAME.replace('/', '_')}_weights.pt"
39     "history_file": "history_tuned_SpaceInvaders.json",
40 },
41 ]
42
43
44 def build_agent_for_test(agent_class, model_class):
45     """Crea un agente listo para test (memoria mínima, epsilon de test
46     model = model_class(N_ACTIONS, WINDOW_LENGTH).to(device)
47     memory = ReplayMemory(capacity=1000)
48     policy = LinearAnnealedEpsGreedy(
49         value_max=1.0, value_min=0.1, value_test=0.05, nb_steps=1
50     )
51     return agent_class(
52         model=model,
53         n_actions=N_ACTIONS,
54         memory=memory,
55         policy=policy,
56         device=device,
57     )
58
59
60 # -----
61 # 1. Carga de modelos + test + carga de históricos
62 # -----
63 results = {}
64 histories = {}
65
66 for cfg in MODELS_TO_COMPARE:
67     name = cfg["name"]
68
69     # --- Test con pesos entrenados ---
70     if os.path.exists(cfg["weights_file"]):
71         print(f"\n{'='*15} Evaluando: {name} {'='*15}")
72         agent = build_agent_for_test(cfg["agent_class"], cfg["model_cla
73         agent.load_weights(cfg["weights_file"])
74
75         processor = AtariProcessor(INPUT_SHAPE, WINDOW_LENGTH)
76         rewards, mean_reward = test(

```

```

77         agent, processor, ENV_NAME, n_episodes=N_TEST_EPISODES, ren
78     )
79     results[name] = {
80         "rewards": rewards,
81         "mean_reward": mean_reward,
82         "std_reward": float(np.std(rewards)),
83     }
84     else:
85         print(f"[AVISO] No se encontraron pesos para '{name}' en {cfg['
86
87     # --- Histórico de entrenamiento ---
88     if os.path.exists(cfg["history_file"]):
89         with open(cfg["history_file"], "r", encoding="utf-8") as f:
90             histories[name] = json.load(f)
91     else:
92         print(f"[AVISO] No se encontró el histórico de '{name}' en {cfg
93
94 # -----
95 # 2. Panel de gráficas de justificación
96 # -----
97 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
98
99 # (a) Curva de aprendizaje: reward medio vs steps
100 for name, h in histories.items():
101     axes[0, 0].plot(h["step"], h["mean_reward_20ep"], linewidth=2, labe
102 axes[0, 0].set_title("Evolución del reward durante el entrenamiento")
103 axes[0, 0].set_xlabel("Steps")
104 axes[0, 0].set_ylabel("Reward medio (últ. 20 ep)")
105 axes[0, 0].legend()
106 axes[0, 0].grid(True, alpha=0.3)
107
108 # (b) Q-valor medio vs steps (detecta sobreestimación DQN vs DDQN)
109 for name, h in histories.items():
110     if "mean_q" in h and len(h["mean_q"]) > 0:
111         axes[0, 1].plot(h["step"], h["mean_q"], linewidth=2, label=name
112 axes[0, 1].set_title("Evolución del Q-valor medio estimado")
113 axes[0, 1].set_xlabel("Steps")
114 axes[0, 1].set_ylabel("Q-valor")
115 axes[0, 1].legend()
116 axes[0, 1].grid(True, alpha=0.3)
117
118 # (c) Recompensa media en test, +/- desviación típica
119 if results:
120     names_r = list(results.keys())
121     means = [results[n]["mean_reward"] for n in names_r]
122     stds = [results[n]["std_reward"] for n in names_r]
123
124     axes[1, 0].bar(names_r, means, yerr=stds, capsize=6, color="tab:blu
125 axes[1, 0].set_title(f"Recompensa media en test ({N_TEST_EPISODES}
126 axes[1, 0].set_ylabel("Recompensa media")
127 axes[1, 0].grid(True, axis="y", alpha=0.3)
128     for i, m in enumerate(means):
129         axes[1, 0].text(i, m + max(means) * 0.02, f"{m:.1f}", ha="cente
130
131     # (d) Distribución de recompensas por episodio (boxplot)

```

```
132     box_data = [results[n]["rewards"] for n in names_r]
133     axes[1, 1].boxplot(box_data)
134     axes[1, 1].set_xticks(range(1, len(names_r) + 1))
135     axes[1, 1].set_xticklabels(names_r)
136     axes[1, 1].set_title("Distribución de recompensas por episodio")
137     axes[1, 1].set_ylabel("Recompensa por episodio")
138     axes[1, 1].grid(True, axis="y", alpha=0.3)
139 else:
140     axes[1, 0].axis("off")
141     axes[1, 1].axis("off")
142     print("No se ha podido evaluar ningún modelo en test: revisa los fi
143
144 plt.tight_layout()
145 plt.savefig("comparativa_justificacion_modelos.png", dpi=300, bbox_inch
146 plt.show()
```


===== Evaluando: DQN =====

Episodio 1: reward: 595.000, steps: 729
Episodio 2: reward: 440.000, steps: 762
Episodio 3: reward: 390.000, steps: 647
Episodio 4: reward: 340.000, steps: 618
Episodio 5: reward: 315.000, steps: 592
Episodio 6: reward: 470.000, steps: 693
Episodio 7: reward: 480.000, steps: 805
Episodio 8: reward: 365.000, steps: 661
Episodio 9: reward: 460.000, steps: 866
Episodio 10: reward: 470.000, steps: 907
Episodio 11: reward: 395.000, steps: 644
Episodio 12: reward: 600.000, steps: 974
Episodio 13: reward: 410.000, steps: 804
Episodio 14: reward: 285.000, steps: 644
Episodio 15: reward: 330.000, steps: 615
Episodio 16: reward: 545.000, steps: 579
Episodio 17: reward: 275.000, steps: 459
Episodio 18: reward: 420.000, steps: 838
Episodio 19: reward: 295.000, steps: 512
Episodio 20: reward: 295.000, steps: 580

Recompensa media en 20 episodios: 408.750

===== Evaluando: DDQN =====

Episodio 1: reward: 520.000, steps: 960
Episodio 2: reward: 290.000, steps: 569
Episodio 3: reward: 380.000, steps: 570
Episodio 4: reward: 400.000, steps: 629
Episodio 5: reward: 570.000, steps: 1049
Episodio 6: reward: 225.000, steps: 423
Episodio 7: reward: 385.000, steps: 577
Episodio 8: reward: 570.000, steps: 830
Episodio 9: reward: 325.000, steps: 518
Episodio 10: reward: 570.000, steps: 870
Episodio 11: reward: 245.000, steps: 519
Episodio 12: reward: 410.000, steps: 725
Episodio 13: reward: 115.000, steps: 364
Episodio 14: reward: 275.000, steps: 609
Episodio 15: reward: 310.000, steps: 486
Episodio 16: reward: 440.000, steps: 762
Episodio 17: reward: 525.000, steps: 749
Episodio 18: reward: 325.000, steps: 545
Episodio 19: reward: 170.000, steps: 380
Episodio 20: reward: 380.000, steps: 609

Recompensa media en 20 episodios: 371.500

===== Evaluando: Dueling DQN =====

Episodio 1: reward: 320.000, steps: 541
Episodio 2: reward: 305.000, steps: 419
Episodio 3: reward: 390.000, steps: 671
Episodio 4: reward: 405.000, steps: 599
Episodio 5: reward: 295.000, steps: 503
Episodio 6: reward: 395.000, steps: 593
Episodio 7: reward: 540.000, steps: 969
Episodio 8: reward: 370.000, steps: 510
Episodio 9: reward: 470.000, steps: 704
Episodio 10: reward: 685.000, steps: 770

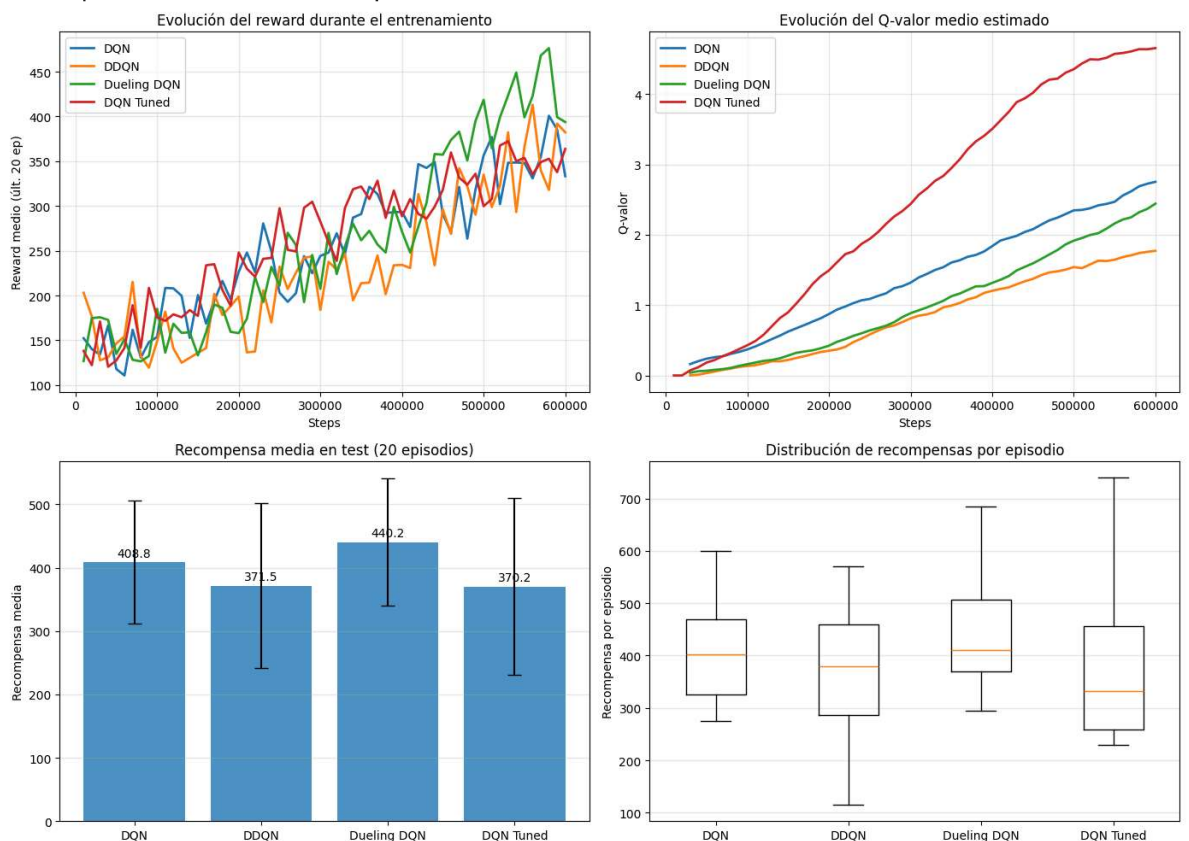
Episodio 11: reward: 465.000, steps: 956
 Episodio 12: reward: 570.000, steps: 807
 Episodio 13: reward: 570.000, steps: 933
 Episodio 14: reward: 370.000, steps: 589
 Episodio 15: reward: 470.000, steps: 752
 Episodio 16: reward: 545.000, steps: 1186
 Episodio 17: reward: 415.000, steps: 781
 Episodio 18: reward: 335.000, steps: 557
 Episodio 19: reward: 495.000, steps: 790
 Episodio 20: reward: 395.000, steps: 659

Recompensa media en 20 episodios: 440.250

===== Evaluando: DQN Tuned =====

Episodio 1: reward: 240.000, steps: 414
 Episodio 2: reward: 245.000, steps: 541
 Episodio 3: reward: 445.000, steps: 616
 Episodio 4: reward: 365.000, steps: 616
 Episodio 5: reward: 500.000, steps: 702
 Episodio 6: reward: 290.000, steps: 453
 Episodio 7: reward: 275.000, steps: 551
 Episodio 8: reward: 490.000, steps: 946
 Episodio 9: reward: 230.000, steps: 488
 Episodio 10: reward: 315.000, steps: 611
 Episodio 11: reward: 255.000, steps: 525
 Episodio 12: reward: 640.000, steps: 877
 Episodio 13: reward: 365.000, steps: 700
 Episodio 14: reward: 260.000, steps: 438
 Episodio 15: reward: 395.000, steps: 733
 Episodio 16: reward: 350.000, steps: 673
 Episodio 17: reward: 740.000, steps: 1033
 Episodio 18: reward: 240.000, steps: 510
 Episodio 19: reward: 500.000, steps: 945
 Episodio 20: reward: 265.000, steps: 547

Recompensa media en 20 episodios: 370.250



3.4.1 - Resultados de entrenamiento

DQN

El DQN mantiene un aprendizaje estable, alcanzando recompensas finales en torno a **350–390**, con picos cercanos a **400**. El Q-valor medio crece hasta aproximadamente **2.8**, reflejando la conocida tendencia del DQN a sobreestimar los valores de acción.

Double DQN

El DDQN presenta un entrenamiento más irregular, aunque termina alrededor de **380–390** puntos. El Q-valor medio final es el más bajo (**≈ 1.8**), confirmando que reduce la sobreestimación respecto al DQN.

Dueling DQN

El Dueling DQN obtiene la mejor evolución durante el entrenamiento, alcanzando recompensas próximas a **400** con un pico cercano a **480**. Su Q-valor medio final ronda **2.4–2.5**, situándose entre DQN y DDQN.

DQN Tuned

El DQN Tuned muestra un entrenamiento muy estable, finalizando alrededor de **360–390** puntos. Sin embargo, el Q-valor medio crece hasta **≈ 4.7** , muy por encima del resto, indicando una fuerte sobreestimación pese a su buen rendimiento.

3.4.2 - Resultados de test (20 episodios, $\epsilon = 0.05$)

Modelo	Recompensa media
Dueling DQN	440.2
DQN	408.8
DDQN	371.5
DQN Tuned	370.2

Los diagramas de caja muestran además que el Dueling DQN posee la mediana más alta y una distribución desplazada hacia recompensas superiores, aunque con cierta variabilidad. El DQN mantiene un comportamiento sólido y consistente. DDQN y DQN Tuned presentan medias similares, siendo el Tuned el de mayor dispersión, con episodios muy buenos pero también otros claramente inferiores.

3.4.3 - Discusión

Los nuevos resultados modifican las conclusiones anteriores. El **Dueling DQN** pasa a ser el mejor modelo tanto durante el entrenamiento como en la evaluación final, alcanzando la mayor recompensa media (**440.2**).